AGENTOS STUDIO

One binary for a fleet of coding agents

YEAR	ROLE	TEAM	STATUS
2026	Product, architecture, full-stack engineering	Solo build	Public preview

Summary

A local control plane for a multi-agent fleet: one Rust binary is both a desktop app and the MCP hub the agents call, with a shared bus, one memory, one run timeline and a merge gate that waits for the checks.

The problem

A fleet of coding agents fails in the gaps between the agents. One tool holds the task queue, another holds the memory, a third holds the pull requests, and nobody can answer who did what, when, and on whose instruction.

AgentOS Studio puts that record in one place on the machine. The same binary is the window a person uses and the hub the agents call, so the queue, the memory, the worktrees and the audit trail cannot drift apart.

What is built

- One Rust binary that serves the Tauri desktop window and the agentos-hub MCP server with 13 tools over JSON-RPC 2.0.
- Missions with an isolated git worktree and a run timeline that records every bus line, tool call, review and done status.
- A Brain view over the Obsidian vault with typed notes, link counts and recall ranked by match, recency and link in-degree.
- An agent registry with start commands, cost models, quota headroom and measured mission outcomes.
- Local SQLite storage, credentials in the OS keychain and no telemetry.

Decisions that matter

One binary, two roles

Choice. The desktop window and the agentos-hub MCP server ship as one Rust program, with the server started from the same executable.

Why. Two programs drift: the window shows a queue the agents do not use, or the agents write a record the window cannot read.

Trade-off. Every change to the hub is a change to the desktop app, so both ship on the same release.

A local record, no telemetry

Choice. State lives in SQLite and the Obsidian vault on the machine, and credentials stay in the operating system keychain.

Why. The record holds client repositories, branch names and agent conversations. Hosting it would create a breach target without making the fleet better.

Trade-off. No hosted dashboard, and backups are the owner's job.

Merge only after the checks

Choice. Missions move through plan, worktree, review, merge and verify, and a merge is refused when its checks have not passed.

Why. An agent that merges without evidence turns a small mistake into a broken main branch.

Trade-off. Every merge waits for CI.

Stack

`Rust` `Tauri 2` `React 19` `TypeScript` `SQLite` `MCP`

Where to look

- Source on GitHub: https://github.com/konethegreat/agentos-studio
- Architecture docs: https://github.com/konethegreat/agentos-studio/blob/main/docs/ARCHITECTURE.md
- Case study: konethegreat.github.io/work/agentos-studio